

22.02.2026, AALEN

MACHINE LEARNING AND DEEP LEARNING

PROJECT 1; GROUP 4

QR Code & Link zum Google Colab unseres Projekts



Generiert mit Nano Banana



<https://colab.research.google.com/drive/1yYsZch0dEkwV8WvkbjrFEj5WRPeidCT#scrollTo=IV895LrHgFbx>



Generiert mit Nano Banana

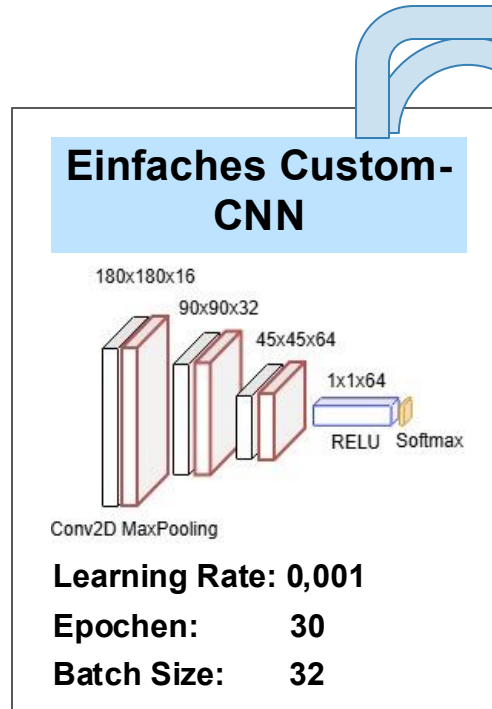
- 01** Start mit Experimentieren:
Erkenntnisse anhand eines Custom CNN
- 02** Problematik des **Imbalanced Datasets**
- 03** Maßnahme 1: **Data Augmentation**
- 04** Maßnahme 2: **Class Weights**
- 05** Modell-Tuning
- 06** Confusion matrix
- 07** *Variation A:* **Classifier Ensemble**
- 08** *Variation B:* **2-Head-Classifier**
- 09** Pretrained Models
Vergleich: 3-Head-Classfier vs 3 Einzelmodelle



Generiert mit Nano Banana

- 01** **Start mit Experimentieren:**
Erkenntnisse anhand eines Custom CNN
- 02 Problematik des **Imbalanced Datasets**
- 03 Maßnahme 1: **Data Augmentation**
- 04 Maßnahme 2: **Class Weights**
- 05 Modell-Tuning
- 06 Confusion matrix
- 07 *Variation A:* **Classifier Ensemble**
- 08 *Variation B:* **2-Head-Classifier**
- 09 Pretrained Models
Vergleich: 3-Head-Classifier vs 3 Einzelmodelle

01 Start mit Experimentieren - Erkenntnisse anhand eines einfachen Custom CNN



Kategorie-Klassifikation

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut	51	51	1.0	1.0	1.0	1.0
1	screw	46	46	1.0	1.0	1.0	1.0
2	pill	43	43	1.0	1.0	1.0	1.0
3	carpet	41	41	1.0	1.0	1.0	1.0
4	zipper	41	41	1.0	1.0	1.0	1.0
5	leather	38	38	1.0	1.0	1.0	1.0
6	cable	36	36	1.0	1.0	1.0	1.0
7	tile	36	36	1.0	1.0	1.0	1.0
8	capsule	34	34	1.0	1.0	1.0	1.0
9	wood	33	33	1.0	1.0	1.0	1.0
10	grid	33	33	1.0	1.0	1.0	1.0
11	metal_nut	32	32	1.0	1.0	1.0	1.0
12	transistor	31	31	1.0	1.0	1.0	1.0
13	bottle	29	29	1.0	1.0	1.0	1.0
14	toothbrush	10	10	1.0	1.0	1.0	1.0
15	ALL	534	534	1.0	1.0	1.0	1.0

Alle Klassen werden
perfekt erkannt

Kategorie + (IO/NIO)-Klassifikation

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_IO	43	50	0.860000	1.000000	0.924731	0.986891
1	screw_IO	36	46	0.782609	1.000000	0.878049	0.981273
2	carpet_IO	31	41	0.756098	1.000000	0.861111	0.981273
3	pill_IO	29	43	0.674419	1.000000	0.805556	0.973783
4	grid_IO	28	33	0.848485	1.000000	0.918033	0.990637
5	leather_IO	28	36	0.777778	1.000000	0.875000	0.985019
6	cable_IO	28	31	0.806452	0.892857	0.847458	0.983146
7	wood_IO	27	33	0.818182	1.000000	0.900000	0.988764
8	transistor_IO	27	30	0.900000	1.000000	0.947368	0.994382
9	zipper_IO	27	41	0.658537	1.000000	0.794118	0.973783
10	tile_IO	26	33	0.757576	0.961538	0.847458	0.983146
11	metal_nut_IO	24	30	0.800000	1.000000	0.888889	0.988764
12	capsule_IO	24	34	0.705882	1.000000	0.827586	0.981273
13	bottle_IO	23	27	0.851852	1.000000	0.920000	0.992509
14	zipper_NIO	14	0	0.000000	0.000000	0.000000	0.973783
15	pill_NIO	14	0	0.000000	0.000000	0.000000	0.973783
16	tile_NIO	10	3	0.666667	0.200000	0.307692	0.983146
17	screw_NIO	10	0	0.000000	0.000000	0.000000	0.981273
18	carpet_NIO	10	0	0.000000	0.000000	0.000000	0.981273
19	capsule_NIO	10	0	0.000000	0.000000	0.000000	0.981273
20	leather_NIO	10	2	1.000000	0.200000	0.333333	0.985019
21	hazelnut_NIO	8	1	1.000000	0.125000	0.222222	0.986891
22	cable_NIO	8	5	0.400000	0.250000	0.307692	0.983146
23	metal_nut_NIO	8	2	1.000000	0.250000	0.400000	0.988764
24	toothbrush_IO	7	10	0.700000	1.000000	0.823529	0.994382
25	bottle_NIO	6	2	1.000000	0.333333	0.500000	0.992509
26	wood_NIO	6	0	0.000000	0.000000	0.000000	0.988764
27	grid_NIO	5	0	0.000000	0.000000	0.000000	0.990637
28	transistor_NIO	4	1	1.000000	0.250000	0.400000	0.94382
29	toothbrush_NIO	3	0	0.000000	0.000000	0.000000	0.994382
30	ALL	534	534	0.592151	0.548758	0.517661	0.779026

Defekte (NIO) werden
kaum vorhergesagt

Der F1-Score von 0.52
weist auf Problem hin

Die 0.78 Accuracy
scheint trügerisch zu
sein



Generiert mit Nano Banana

- 01 Start mit Experimentieren:
Erkenntnisse anhand eines Custom CNN
- 02 Problematik des **Imbalanced Datasets**
- 03 Maßnahme 1: **Data Augmentation**
- 04 Maßnahme 2: **Class Weights**
- 05 Modell-Tuning
- 06 Confusion matrix
- 07 *Variation A:* **Classifier Ensemble**
- 08 *Variation B:* **2-Head-Classifier**
- 09 Pretrained Models
Vergleich: 3-Head-Classifier vs 3 Einzelmodelle

02 Problematik des Imbalanced Datasets

Die ungleiche Verteilung "OK" und "Fehler" muss behandelt werden.

Im Dataset sind fehlerhafte Bauteile unterrepräsentiert

	Kategorie	Anteil Kategorie (%)	Gesamtanzahl	Anzahl Ok	Anzahl Fehler	Anteil Fehler (%)	Anteil Ok (%)
0	bottle	5 %	292	229	63	22 %	78 %
1	cable	7 %	374	282	92	25 %	75 %
2	capsule	7 %	351	242	109	31 %	69 %
3	carpet	7 %	397	308	89	22 %	78 %
4	grid	6 %	342	285	57	17 %	83 %
5	hazelnut	9 %	501	431	70	14 %	86 %
6	leather	7 %	369	277	92	25 %	75 %
7	metal_nut	6 %	335	242	93	28 %	72 %
8	pill	8 %	434	293	141	32 %	68 %
9	screw	9 %	480	361	119	25 %	75 %
10	tile	6 %	347	263	84	24 %	76 %
11	toothbrush	2 %	102	72	30	29 %	71 %
12	transistor	6 %	313	273	40	13 %	87 %
13	wood	6 %	326	266	60	18 %	82 %
14	zipper	7 %	391	272	119	30 %	70 %
15	All	100 %	5354	4096	1258	23 %	77 %

- sehr stark unterrepräsentiert (<20%)
- stark unterrepräsentiert (<30%)
- unterrepräsentiert (30-35%)

Modell neigt dazu auf "OK" zu raten und erreicht damit hohe Accuracy

Prinzipielle Möglichkeiten um Imbalance zu überwinden:

Upsampling durch Data Augmentation
erzeugen von zusätzlichen, künstlichen, aber realistischen "Fehler"-Bildern bis ca. 50% Verhältnis erreicht ist)

weiterverfolgt

Class Weights

Loss-Funktion wird beim Training mit dem Verhältnis die vorhandenen Klassen gewichtet.

weiterverfolgt

Downsampling

reduzieren von "OK"-Bildern im Training bis ca 50% Verhältnis erreicht ist

Modell auf F1-Score optimieren

auf Balanced Accuracy optimieren

weiterverfolgt



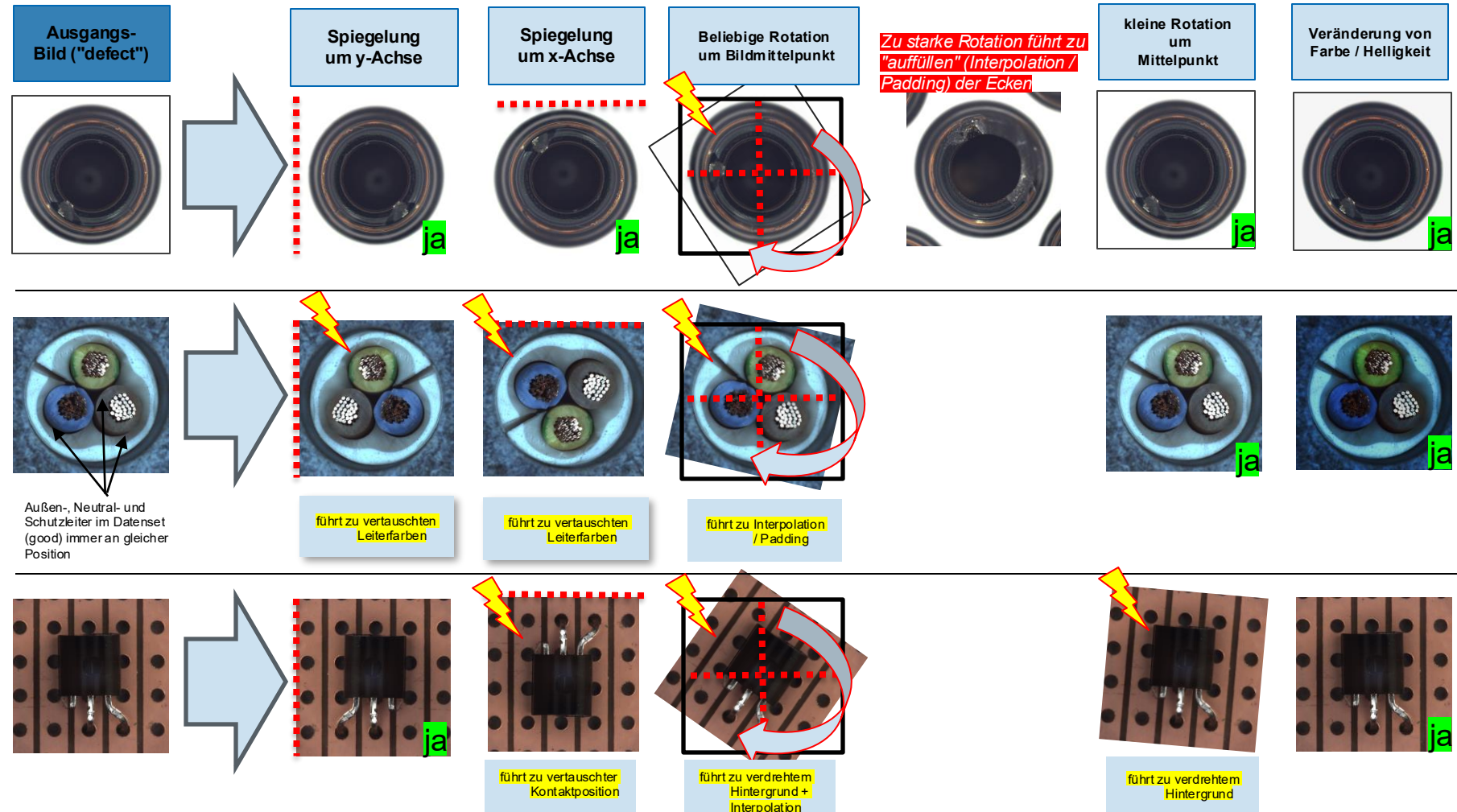
Generiert mit Nano Banana

- 01 Start mit Experimentieren:
Erkenntnisse anhand eines Custom CNN
- 02 Problematik des **Imbalanced Datasets**
- 03** Maßnahme 1: **Data Augmentation**
- 04 Maßnahme 2: **Class Weights**
- 05 Modell-Tuning
- 06 Confusion matrix
- 07 *Variation A: Classifier Ensemble*
- 08 *Variation B: 2-Head-Classifier*
- 09 Pretrained Models
Vergleich: 3-Head-Classifier vs 3 Einzelmodelle

03 Data Augmentation

Erweiterung der unterrepräsentierten Klasse durch zusätzliche, modifizierte Bilder

- **Konzept:** Durch Kopie und Manipulation werden zusätzliche Bilder der unterrepräsentierten Klasse (defekt) erzeugt und für Training verwendet
- **Vorgehensweise:** Bilder wurden spezifisch für jede Produktart beurteilt, welche Manipulation möglich ist, um ein realistisches Augmentat zu erhalten.



03 Data Augmentation

F1 Score erhöht sich durch Augmentation

- **Konzept:** Durch Kopie und Manipulation werden zusätzliche Bilder der unterrepräsentierten Klasse (Defekt) erzeugt und für das Training verwendet.
- **Vorgehensweise:** Bilder wurden spezifisch für jede Produktart beurteilt, welche Manipulation möglich ist, um ein realistisches Augmentat zu erhalten. Danach wurden so viele Augmentate der Klasse (Defekt) erzeugt bis Gleichgewicht vor dem Training herrscht.

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	36	1.000000	0.720000	0.837209	0.982948
1	hazelnut_IO	43	57	0.754386	1.000000	0.860000	0.982948
2	screw_IO	36	44	0.659091	0.805556	0.725000	0.973203
3	carpet_NIO	35	35	0.742857	0.742857	0.742857	0.978076
4	tile_NIO	32	25	1.000000	0.781250	0.877193	0.991474
5	carpet_IO	31	31	0.709677	0.709677	0.709677	0.978076
6	zipper_NIO	31	29	0.931034	0.870968	0.900000	0.992692
7	screw_NIO	31	23	0.695652	0.516129	0.592593	0.973203
8	leather_NIO	30	32	0.843750	0.900000	0.870968	0.990256
9	pill_IO	29	15	0.866667	0.448276	0.590909	0.978076
10	leather_IO	28	26	0.884615	0.821429	0.851852	0.990256
11	grid_IO	28	53	0.509434	0.964286	0.666667	0.967113
12	pill_NIO	28	42	0.619048	0.928571	0.742857	0.978076
13	cable_IO	28	29	0.896552	0.928571	0.912281	0.993910
14	wood_NIO	27	27	0.592593	0.592593	0.592593	0.973203
15	zipper_IO	27	29	0.862069	0.925926	0.892857	0.992692
16	transistor_IO	27	38	0.710526	1.000000	0.830769	0.986602
17	wood_IO	27	27	0.592593	0.592593	0.592593	0.973203
18	transistor_NIO	27	16	1.000000	0.592593	0.744186	0.986602
19	tile_IO	26	29	0.896552	1.000000	0.945455	0.996346
20	grid_NIO	25	4	0.750000	0.120000	0.206897	0.971985
21	cable_NIO	25	24	0.916667	0.880000	0.897959	0.993910
22	metal_nut_IO	24	27	0.814815	0.916667	0.862745	0.991474
23	capsule_IO	24	34	0.647059	0.916667	0.758621	0.982948
24	bottle_IO	23	28	0.821429	1.000000	0.901961	0.993910
25	capsule_NIO	22	12	0.833333	0.454545	0.588235	0.982948
26	bottle_NIO	22	17	1.000000	0.772727	0.871795	0.993910
27	metal_nut_NIO	22	19	0.894737	0.772727	0.829268	0.991474
28	toothbrush_IO	7	8	0.875000	1.000000	0.933333	0.998782
29	toothbrush_NIO	6	5	1.000000	0.833333	0.909091	0.998782
30	ALL	821	821	0.810671	0.783598	0.774614	0.779537

Defekte (NIO) werden nun häufiger und annähernd der Verteilung im Datenset vorhergesagt

Der **F1-Score** hat sich von 0.52 auf 0.77 stark verbessert

Die **Accuracy** bleibt bei 0.78



Generiert mit Nano Banana

- 01 Start mit Experimentieren:
Erkenntnisse anhand eines Custom CNN
- 02 Problematik des **Imbalanced Datasets**
- 03 Maßnahme 1: **Data Augmentation**
- 04** Maßnahme 2: **Class Weights**
- 05 Modell-Tuning
- 06 Confusion matrix
- 07 *Variation A:* **Classifier Ensemble**
- 08 *Variation B:* **2-Head-Classifier**
- 09 Pretrained Models
Vergleich: 3-Head-Classifier vs 3 Einzelmodelle

04 Class Weights

Durch Klassengewichte wird beim Training ein Ausgleich für die Imbalance geschaffen

- **Konzept:** Anpassung der Loss-Funktion während des Trainings.
- **Funktionsweise:** Die Klassenhäufigkeit wird spezifisch für jede Klasse im Trainingsdatensatz ermittelt und im Training übergeben. Unterrepräsentierte Klasse (Defekt) erhalten ein höheres Gewicht durch die Fehlerfunktion
- **Vorteil:** Keine physische Kopie von Daten wie bei Augmentation nötig, mathematische Balance.

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_IO	43	44	0.954545	0.976744	0.965517	0.994382
1	screw_IO	36	35	0.800000	0.777778	0.788732	0.971910
2	carpet_IO	31	27	0.814815	0.709677	0.758621	0.973783
3	pill_IO	29	15	0.866667	0.448276	0.590909	0.966292
4	grid_IO	28	29	0.862069	0.892857	0.877193	0.986891
5	leather_IO	28	24	0.958333	0.821429	0.884615	0.988764
6	cable_IO	28	28	0.857143	0.857143	0.857143	0.985019
7	wood_IO	27	18	0.888889	0.592593	0.711111	0.975655
8	transistor_IO	27	29	0.931034	1.000000	0.964286	0.996255
9	zipper_IO	27	28	0.892857	0.925926	0.909091	0.990637
10	tile_IO	26	31	0.806452	0.961538	0.877193	0.985019
11	metal_nut_IO	24	26	0.846154	0.916667	0.880000	0.988764
12	capsule_IO	24	26	0.807692	0.875000	0.840000	0.985019
13	bottle_IO	23	24	0.958333	1.000000	0.978723	0.998127
14	zipper_NIO	14	13	0.846154	0.785714	0.814815	0.990637
15	pill_NIO	14	28	0.428571	0.857143	0.571429	0.966292
16	tile_NIO	10	4	0.750000	0.300000	0.428571	0.985019
17	screw_NIO	10	11	0.272727	0.300000	0.285714	0.971910
18	carpet_NIO	10	14	0.357143	0.500000	0.416667	0.973783
19	capsule_NIO	10	8	0.625000	0.500000	0.555556	0.985019
20	leather_NIO	10	14	0.642857	0.900000	0.750000	0.988764
21	hazelnut_NIO	8	7	0.857143	0.750000	0.800000	0.994382
22	cable_NIO	8	8	0.500000	0.500000	0.500000	0.985019
23	metal_nut_NIO	8	6	0.666667	0.500000	0.571429	0.988764
24	toothbrush_IO	7	7	0.857143	0.857143	0.857143	0.996255
25	bottle_NIO	6	5	1.000000	0.833333	0.909091	0.998127
26	wood_NIO	6	15	0.266667	0.666667	0.380952	0.975655
27	grid_NIO	5	5	0.400000	0.400000	0.400000	0.988764
28	transistor_NIO	4	2	1.000000	0.500000	0.666667	0.996255
29	toothbrush_NIO	3	3	0.666667	0.666667	0.666667	0.996255
30	ALL	534	534	0.746057	0.719076	0.715261	0.784644

Defekte (NIO) werden nun häufiger und annähernd der Verteilung im Datensatz vorhergesagt

Der **F1-Score** hat sich von 0.52 auf 0.72 **auch deutlich verbessert**, jedoch nicht ganz so stark wie bei Augmentierung (0.78)

Die **Accuracy** bleibt bei 0.78



Generiert mit Nano Banana

- 01 Start mit Experimentieren:
Erkenntnisse anhand eines Custom CNN
- 02 Problematik des **Imbalanced Datasets**
- 03 Maßnahme 1: **Data Augmentation**
- 04 Maßnahme 2: **Class Weights**
- 05** Modell-Tuning
- 06 Confusion matrix
- 07 *Variation A:* **Classifier Ensemble**
- 08 *Variation B:* **2-Head-Classifier**
- 09 Pretrained Models
Vergleich: 3-Head-Classfier vs 3 Einzelmodelle

05 Modell-Tuning

Tuning Übersicht

- Tuning des einfachen CNNs
- Erster Gedanke: Autotune mit Keras
 - Wir möchten 7 Hyperparameter untersuchen
 - Eine kombinatorische Hyperparametersuche würde zu einer hohen Anzahl an Trainings führen
- Lösung:
 - Wir passen immer nur 1nen Hyperparameter an
 - Pro Durchlauf Vergleich des Modells mit dem Basismodell
 - Beschränkung auf 15 Epochen

Ergebnisse

- **Tuning-Ergebnis:**

Vereinzelt führen folgende Hyperparameter zu einem besseren F1-Score als das Basismodell

 - 4 Conv-Blöcke
 - Dense-Layer von 128
- Wir haben uns dazu entschieden, für das getunte Modell das Basismodell mit beiden Hyperparameter zu verwenden.




	name	blocks	filters	use_gap	dense_layers	dense_units	lr	accuracy	f1
0	blocks_4	4	medium	False	1	64	0.0003	0.803898	0.799060
1	dense_128	3	medium	False	1	128	0.0003	0.808770	0.796494
2	baseline	3	medium	False	1	64	0.0003	0.778319	0.768647
3	blocks_2	2	medium	False	1	64	0.0003	0.773447	0.756934
4	dense_layers_2	3	medium	False	2	64	0.0003	0.755177	0.737358
5	filters_small	3	small	False	1	64	0.0003	0.750305	0.733872
6	filters_large	3	large	False	1	64	0.0003	0.728380	0.704806
7	use_gap	3	medium	True	1	64	0.0003	0.641900	0.535766

05 Modell-Tuning

Ungetuntetes Modell

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	36	1.000000	0.720000	0.837209	0.982948
1	hazelnut_IO	43	57	0.754386	1.000000	0.860000	0.982948
2	screw_IO	36	44	0.659091	0.805556	0.725000	0.973203
3	carpet_NIO	35	35	0.742857	0.742857	0.742857	0.978076
4	tile_NIO	32	25	1.000000	0.781250	0.877193	0.991474
5	carpet_IO	31	31	0.709677	0.709677	0.709677	0.978076
6	zipper_NIO	31	29	0.931034	0.870968	0.900000	0.992692
7	screw_NIO	31	23	0.695652	0.516129	0.592593	0.973203
8	leather_NIO	30	32	0.843750	0.900000	0.870968	0.990256
9	pill_IO	29	15	0.866667	0.448276	0.590909	0.978076
10	leather_IO	28	26	0.884615	0.821429	0.851852	0.990256
11	grid_IO	28	53	0.509434	0.964286	0.666667	0.967113
12	pill_NIO	28	42	0.619048	0.928571	0.742857	0.978076
13	cable_IO	28	29	0.896552	0.928571	0.912281	0.993910
14	wood_NIO	27	27	0.592593	0.592593	0.592593	0.973203
15	zipper_IO	27	29	0.862069	0.925926	0.892857	0.992692
16	transistor_IO	27	38	0.710526	1.000000	0.830769	0.986602
17	wood_IO	27	27	0.592593	0.592593	0.592593	0.973203
18	transistor_NIO	27	16	1.000000	0.592593	0.744186	0.986602
19	tile_IO	26	29	0.896552	1.000000	0.945455	0.996346
20	grid_NIO	25	4	0.750000	0.120000	0.206897	0.971985
21	cable_NIO	25	24	0.916667	0.880000	0.897959	0.993910
22	metal_nut_IO	24	27	0.814815	0.916667	0.862745	0.991474
23	capsule_IO	24	34	0.647059	0.916667	0.758621	0.982948
24	bottle_IO	23	28	0.821429	1.000000	0.901961	0.993910
25	capsule_NIO	22	12	0.833333	0.454545	0.588235	0.982948
26	bottle_NIO	22	17	1.000000	0.772727	0.871795	0.993910
27	metal_nut_NIO	22	19	0.894737	0.772727	0.829268	0.991474
28	toothbrush_IO	7	8	0.875000	1.000000	0.933333	0.99782
29	toothbrush_NIO	6	5	1.000000	0.833333	0.909091	0.998782
30	ALL	821	821	0.810671	0.783598	0.774614	0.779537

0,7746

Getuntetes Modell

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	44	0.977273	0.860000	0.914894	0.990256
1	hazelnut_IO	43	49	0.857143	0.976744	0.913043	0.990256
2	screw_IO	36	29	0.620690	0.500000	0.553846	0.964677
3	carpet_NIO	35	36	0.777778	0.800000	0.788732	0.981730
4	tile_NIO	32	11	0.909091	0.312500	0.465116	0.971985
5	carpet_IO	31	30	0.766667	0.741935	0.754098	0.981730
6	zipper_NIO	31	28	0.964286	0.870968	0.915254	0.993910
7	screw_NIO	31	38	0.526316	0.645161	0.579710	0.964677
8	leather_NIO	30	29	0.931034	0.900000	0.915254	0.993910
9	pill_IO	29	40	0.675000	0.931034	0.782609	0.981730
10	leather_IO	28	29	0.896552	0.928571	0.912281	0.993910
11	grid_IO	28	28	0.821429	0.821429	0.821429	0.987820
12	pill_NIO	28	17	0.882353	0.535714	0.666667	0.981730
13	cable_IO	28	34	0.705882	0.857143	0.774194	0.982948
14	wood_NIO	27	21	0.952381	0.740741	0.833333	0.990256
15	zipper_IO	27	30	0.866667	0.962963	0.912281	0.993910
16	transistor_IO	27	30	0.900000	1.000000	0.947368	0.996346
17	wood_IO	27	33	0.787879	0.962963	0.866667	0.990256
18	transistor_NIO	27	24	1.000000	0.888889	0.941176	0.996346
19	tile_IO	26	43	0.581395	0.961538	0.724638	0.976857
20	grid_NIO	25	29	0.689655	0.800000	0.740741	0.982948
21	cable_NIO	25	19	0.789474	0.600000	0.681818	0.982948
22	metal_nut_IO	24	28	0.821429	0.958333	0.884615	0.992692
23	capsule_IO	24	31	0.741935	0.958333	0.836364	0.989038
24	bottle_IO	23	27	0.851852	1.000000	0.920000	0.995128
25	capsule_NIO	22	15	0.933333	0.636364	0.756757	0.989038
26	bottle_NIO	22	18	1.000000	0.818182	0.900000	0.995128
27	metal_nut_NIO	22	18	0.944444	0.772727	0.850000	0.992692
28	toothbrush_IO	7	10	0.700000	1.000000	0.823529	0.996346
29	toothbrush_NIO	6	3	1.000000	0.500000	0.666667	0.996346
30	ALL	821	821	0.829065	0.808074	0.801436	0.808770

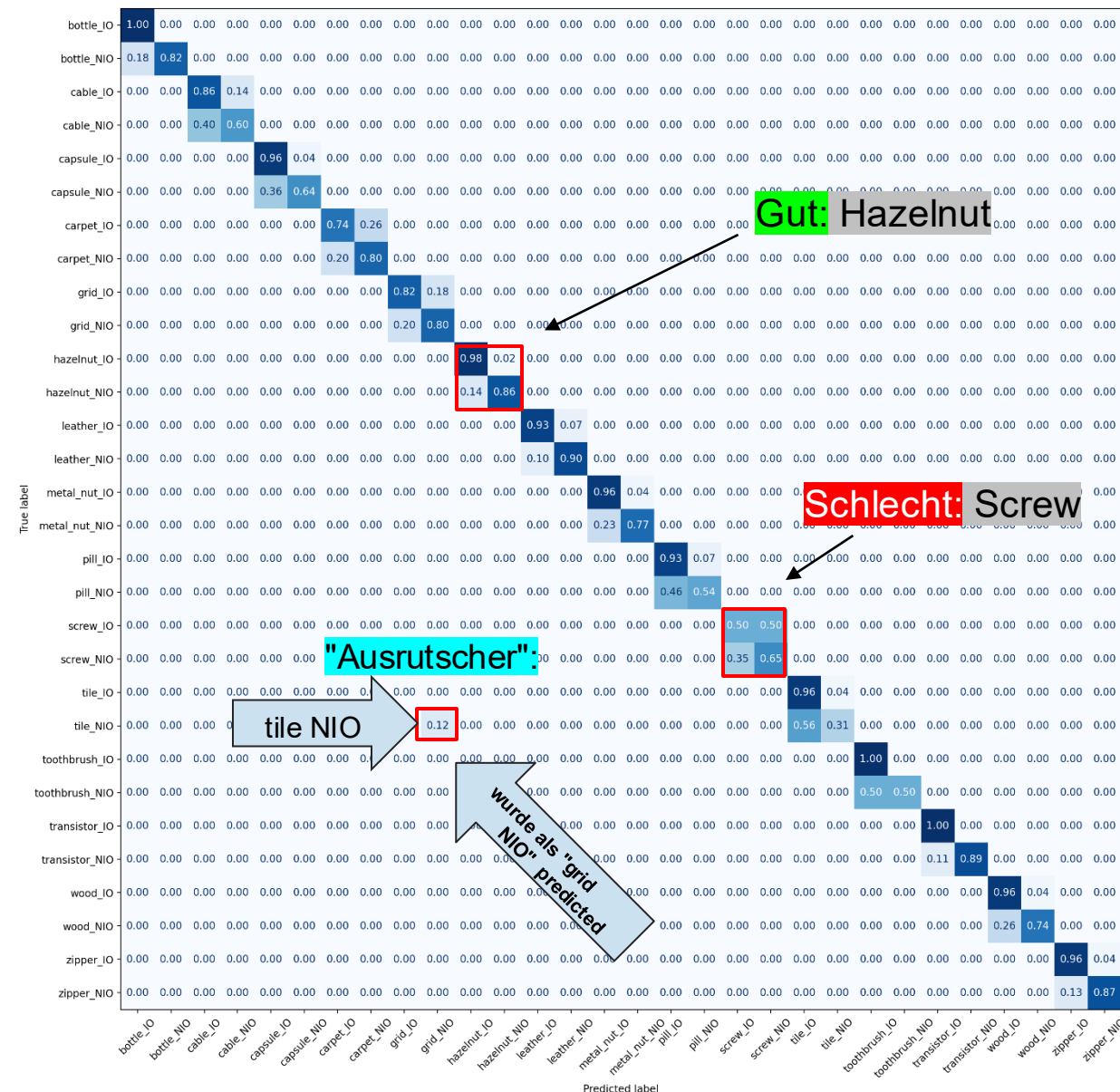
0,801436



Generiert mit Nano Banana

- 01 Start mit Experimentieren:
Erkenntnisse anhand eines Custom CNN
- 02 Problematik des **Imbalanced Datasets**
- 03 Maßnahme 1: **Data Augmentation**
- 04 Maßnahme 2: **Class Weights**
- 05 Modell-Tuning
- 06 Confusion matrix**
- 07 *Variation A: Classifier Ensemble*
- 08 *Variation B: 2-Head-Classifier*
- 09 Pretrained Models
Vergleich: 3-Head-Classifier vs 3 Einzelmodelle

06 Confusion Matrix



Interpretierung

- Zeigt die Verwechslungen des Modells zwischen den Klassen
- Zeile = True Label, Spalte = Predicted Label
- Normalisiert auf 0-1

Bsp. Hazelnut IO Zeile:

- 98 % werden auch als Hazelnut IO vorhergesagt
- 2% werden aber als Hazelnut NIO vorhergesagt

Takeaways

- Dominante Diagonale → Kategorien meist korrekt vorhergesagt
- Fehler liegen oft innerhalb einer Produktkategorie (IO ↔ NIO)
- Hauptursache: wenige NIO-Beispiele im Ursprungsdatensatz → unsichere Entscheidungen



Generiert mit Nano Banana

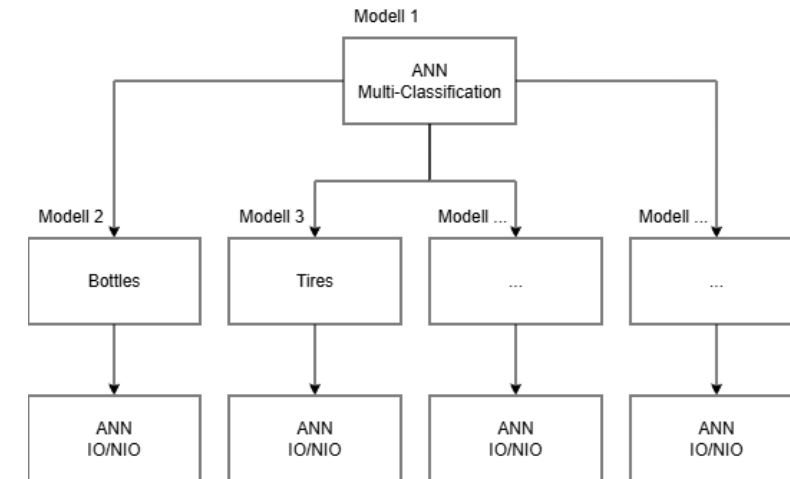
- 01 Start mit Experimentieren:
Erkenntnisse anhand eines Custom CNN
- 02 Problematik des Imbalanced Datasets
- 03 Maßnahme 1: Data Augmentation
- 04 Maßnahme 2: Class Weights
- 05 Modell-Tuning
- 06 Confusion matrix
- 07 Variation A: Classifier Ensemble**
- 08 Variation B: 2-Head-Classifier
- 09 Pretrained Models
Vergleich: 3-Head-Classifier vs 3 Einzelmodelle

07 Variation A: Classifier Ensemble

Idee: Hierarchische Klassifikation

Modellarchitektur

- 1) Kategorie-Modell (15 Klassen) →
- 2) Pro Kategorie ein binäres IO/NIO-Modell
- Output wird zu 30 Klassen zusammengeführt



Performance

- Validierungsdatensatz: F1 Score: 0,75

Erkenntnisse

- Vorteile:
 - IO/NIO-Modelle sind spezialisiert (Binary-Decision)
- Nachteile:
 - Mehr Modelle = mehr Training



Generiert mit Nano Banana

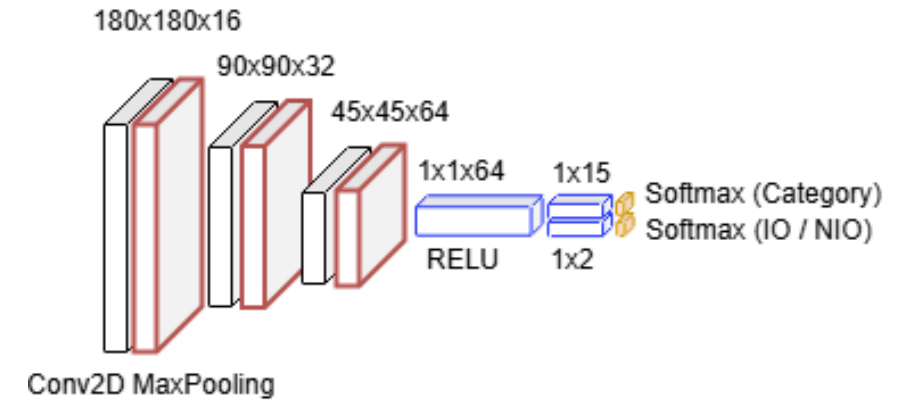
- 01 Start mit Experimentieren:
Erkenntnisse anhand eines Custom CNN
- 02 Problematik des Imbalanced Datasets
- 03 Maßnahme 1: Data Augmentation
- 04 Maßnahme 2: Class Weights
- 05 Modell-Tuning
- 06 Confusion matrix
- 07 *Variation A: Classifier Ensemble*
- 08 *Variation B: 2-Head-Classifier***
- 09 Pretrained Models
Vergleich: 3-Head-Classifer vs 3 Einzelmodelle

08 Variation B: 2-Head-Classifier

Idee: Multitask Learning Ansatz

Modellarchitektur

- Head 1: Kategorie (15 Klassen)
- Head 2: IO/NIO (2 Klassen)
- Mapping der Ausgaben zum 30er Klassifikationsproblem



Performance

- F1 Score: 0,78 (Validierungsdatensatz)

Erkenntnisse

- nur 1 Modell, gemeinsames Feature-Learning



Generiert mit Nano Banana

- 01 Start mit Experimentieren:
Erkenntnisse anhand eines Custom CNN
- 02 Problematik des **Imbalanced Datasets**
- 03 Maßnahme 1: **Data Augmentation**
- 04 Maßnahme 2: **Class Weights**
- 05 Modell-Tuning
- 06 Confusion matrix
- 07 *Variation A:* **Classifier Ensemble**
- 08 *Variation B:* **2-Head-Classifier**
- 09 **Pretrained Models**
Vergleich: 3-Head-Classfier vs 3 Einzelmodelle

09 Pretrained Models - Vergleich: 3-Head-Classifier vs. 3 Einzelmodelle

Set up

3 Heads

- Direkte Klassifikation (Nur 1 Dense mit Softmax)
- Dense Head (2 Dense Layer (RELU), dann 1 Dense Layer mit Softmax)
- 2 Head Two Tasks (1 Dense Layer (RELU), 2 Dense mit Softmax)

3 Backbones (e.g. Pretrained modelle)

- MobileNetV3Small
- Xception
- VGG16

Training

Alle Kombinationen trainiert (3x3=9):

Rank	Backbone	Head	F1
1	Mobile	Direct	0.908737
2	VGG	Direct	0.898162
3	VGG	Dense	0.896013
4	Xception	Direct	0.892971
5	Xception	Dense	0.888092
6	Xception	2-head	0.877179
7	Mobile	Dense	0.872429
8	VGG	2-head	0.872396
9	Mobile	2-head	0.857045

09 Pretrained Models - Vergleich: 3-Head-Classifier vs. 3 Einzelmodelle

Auswahl bestes Modell:

- Mobile Direct (F1: 0.908737)
 - Fine-Tuning:
 - Modell importiert
- 15 Layer des Base-Modells mit minimaler Lernrate weiter trainiert.

Evaluation

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	46	1.000000	0.920000	0.958333	0.995128
1	hazelnut_IO	43	47	0.914894	1.000000	0.955556	0.995128
2	screw_IO	36	41	0.853659	0.972222	0.909091	0.991474
3	carpet_NIO	35	28	0.892857	0.714286	0.793651	0.984166
4	tile_NIO	32	29	1.000000	0.906250	0.950820	0.996346
5	carpet_IO	31	38	0.736842	0.903226	0.811594	0.984166
6	zipper_NIO	31	29	1.000000	0.935484	0.966667	0.997564
7	screw_NIO	31	26	0.961538	0.806452	0.877193	0.991474
8	leather_NIO	30	31	0.967742	1.000000	0.983607	0.998782
9	pill_IO	29	34	0.735294	0.862069	0.793651	0.984166
10	leather_IO	28	27	1.000000	0.964286	0.981818	0.998782
11	grid_IO	28	28	0.821429	0.821429	0.821429	0.987820
12	pill_NIO	28	23	0.826087	0.678571	0.745098	0.984166
13	cable_IO	28	32	0.843750	0.964286	0.900000	0.992692
14	wood_NIO	27	27	1.000000	1.000000	1.000000	1.000000
15	zipper_IO	27	29	0.931034	1.000000	0.964286	0.997564
16	transistor_IO	27	34	0.794118	1.000000	0.885246	0.991474
17	wood_IO	27	27	1.000000	1.000000	1.000000	1.000000
18	transistor_NIO	27	20	1.000000	0.740741	0.851064	0.991474
19	tile_IO	26	29	0.896552	1.000000	0.945455	0.996346
20	grid_NIO	25	25	0.800000	0.800000	0.800000	0.987820
21	cable_NIO	25	21	0.952381	0.800000	0.869565	0.992692
22	metal_nut_IO	24	26	0.923077	1.000000	0.960000	0.997564
23	capsule_IO	24	22	0.954545	0.875000	0.913043	0.995128
24	bottle_IO	23	23	1.000000	1.000000	1.000000	1.000000
25	capsule_NIO	22	24	0.875000	0.954545	0.913043	0.995128
26	bottle_NIO	22	22	1.000000	1.000000	1.000000	1.000000
27	metal_nut_NIO	22	20	1.000000	0.909091	0.952381	0.997564
28	toothbrush_IO	7	6	1.000000	0.857143	0.923077	0.998782
29	toothbrush_NIO	6	7	0.857143	1.000000	0.923077	0.998782
30	ALL	821	821	0.917931	0.912836	0.911625	0.911084

Test

	class	true_count	pred_count	precision	recall	f1	accuracy
0	screw_NIO	45	44	0.818182	0.800000	0.808989	0.978986
1	hazelnut_IO	43	53	0.811321	1.000000	0.895833	0.987639
2	screw_IO	36	37	0.756757	0.777778	0.767123	0.978986
3	hazelnut_NIO	36	26	1.000000	0.722222	0.838710	0.987639
4	cable_NIO	31	26	1.000000	0.838710	0.912281	0.993820
5	carpet_IO	31	30	0.966667	0.935484	0.950820	0.996292
6	pill_IO	30	29	0.862069	0.833333	0.847458	0.988875
7	pill_NIO	30	31	0.838710	0.866667	0.852459	0.988875
8	grid_IO	29	29	0.689655	0.689655	0.689655	0.977750
9	metal_nut_NIO	28	24	1.000000	0.857143	0.923077	0.995056
10	capsule_NIO	28	21	0.952381	0.714286	0.816327	0.988875
11	cable_IO	28	33	0.848485	1.000000	0.918033	0.993820
12	transistor_IO	28	30	0.933333	1.000000	0.965517	0.997528
13	leather_IO	27	27	1.000000	1.000000	1.000000	1.000000
14	zipper_IO	27	26	1.000000	0.962963	0.981132	0.998764
15	wood_NIO	27	23	1.000000	0.851852	0.920000	0.995056
16	transistor_NIO	27	25	1.000000	0.925926	0.961538	0.997528
17	tile_IO	27	27	1.000000	1.000000	1.000000	1.000000
18	leather_NIO	27	27	1.000000	1.000000	1.000000	1.000000
19	wood_IO	26	30	0.866667	1.000000	0.928571	0.995056
20	grid_NIO	25	25	0.640000	0.640000	0.640000	0.977750
21	capsule_IO	24	31	0.741935	0.958333	0.836364	0.988875
22	carpet_NIO	24	25	0.920000	0.958333	0.938776	0.996292
23	metal_nut_IO	24	28	0.857143	1.000000	0.923077	0.995056
24	bottle_IO	23	23	1.000000	1.000000	1.000000	1.000000
25	bottle_NIO	22	22	1.000000	1.000000	1.000000	1.000000
26	tile_NIO	21	21	1.000000	1.000000	1.000000	1.000000
27	zipper_NIO	21	22	0.954545	1.000000	0.976744	0.998764
28	toothbrush_IO	7	7	1.000000	1.000000	1.000000	1.000000
29	toothbrush_NIO	7	7	1.000000	1.000000	1.000000	1.000000
30	ALL	809	809	0.915262	0.911089	0.909749	0.898640

22.02.2026, AALEN

KONTAKTDATEN

SIMON RUTTMANN	80751
MARCEL STAIGER	3011613
TOBIAS KARL	3014537